4

Motor control

When users type on a keyboard, tap on a touchscreen, steer a car, or generally perform an action using their body, they rely on *motor control*. Human motor control refers to the regulation of all movement in a human, including the integration of relevant internal and external sensory information to determine the necessary signals to activate the right muscles. Motor control is critical for fundamental human tasks such as balancing the body, pointing at things, and grasping objects.

Motor control in human–computer interface (HCI) focuses on humans interacting with computing systems, for instance, typing on a keyboard, moving a mouse pointer toward an icon, and reacting to prompts. An in-depth understanding of motor control supports the design of user interfaces that better fit the capabilities of users. Such an understanding is often captured in the form of mathematical or computational models. For example, the average time it takes an average user to touch an app icon on a touchscreen phone can be predicted with a model if we know (1) the distance of the user from the app icon and (2) the size of the app icon. Such predictive models have many applications in HCI, including the following:

1. Graphical layouts can be personalized to better match a user's motor abilities [266]. Depending on the amount of tremor, the elements on an interface can be made bigger and the layout reorganized so that the elements can be reached more easily.
2. Target selection techniques utilize motor control models to dynamically make the objects on display easier to select. For example, they can change the selection areas to support fast selection (Chapter 26).
3. Keyboard layouts have been optimized using models of pointing. For example, letters and characters can be assigned to keys so that the average selection time for a given language is minimized. The Dvorak Simplified Keyboard is a famous early attempt to break the hegemony of the QWERTY layout. While optimized keyboard layouts demonstrate faster and less error-prone typing, this is achieved at the expense of having to learn a new layout [e.g., 70, 239].
4. The performance of input methods and devices can be compared using metrics rooted in models of motor control. For example, we can compare an in-air input method and a fingertip-based input method—two methods that involve widely different movement sizes and types—using the concept of throughput.

Motor control models can also help us understand how difficult certain tasks are for users. Figure 4.1 shows two slightly different examples. Navigating a drop-down menu is surprisingly complex, but Ahlström [12] showed how to do it with two laws that are discussed in this chapter: Fitts' law and the steering law. Before we provide a more in-depth understanding of such examples, let us look at elements of motor control tasks in HCI.

Introduction to Human-Computer Interaction. Kasper Hornbæk, Per Ola Kristensson, and Antti Oulasvirta, Oxford University Press. © Kasper Hornbæk, Per Ola Kristensson, and Antti Oulasvirta (2025).
DOI: 10.1093/oso/9780192864543.003.0004

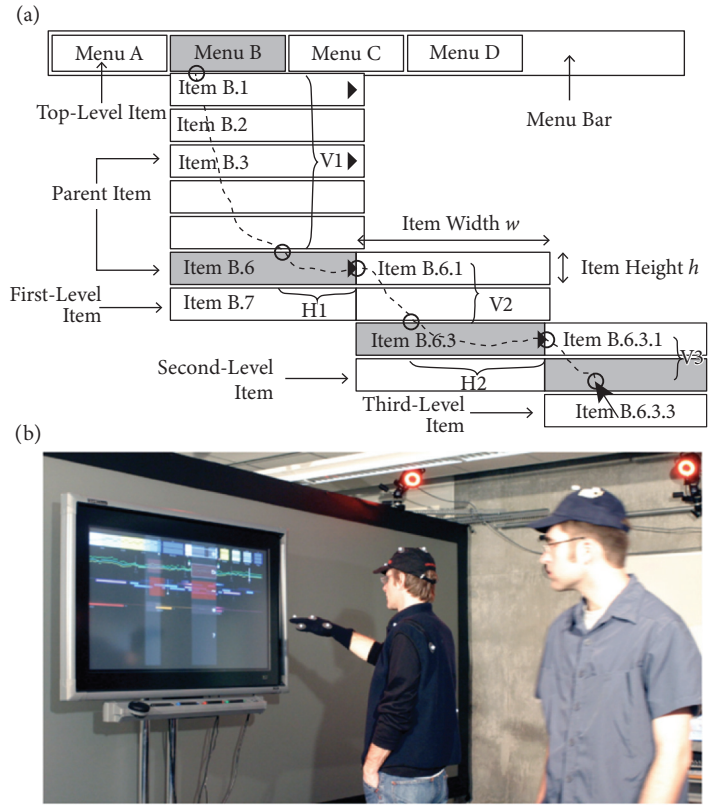

Figure 4.1 Two examples of motor control in HCI. The figure to the top shows how to select an item in a hierarchical drop-down menu. Fitts' law and the steering law can be used to predict users' performance in this task [12]. The figure to the bottom shows a system where the user can point toward a large display from a distance [859] (photo from Vogel's master thesis, p. 51).

4.1 Elements of a motor control task in HCI

Movement control concerns how our nervous system produces purposeful and coordinated movement. In HCI, movement most often occurs through our fingers, although some systems have used movements of the head, feet [29], or the entire body. Even if the movement is done by multiple body parts, in HCI, we are typically only interested in the part we use for something: the *end-effector*.

End-effectors controlled by different body parts have different *degrees of freedom*. For instance, the knee has one degree of freedom (it is basically a hinge), whereas the hip has three degrees of freedom (it can rotate around three axes). These degrees of freedom are often used to define an end-effector in 3D space and rotate it along three axes: roll, pitch, and yaw. This gives a total of six degrees of freedom.

Movement is often complicated. Joints need to work together to produce movement, the environment's effect on the body varies, and the level of muscle fatigue also varies. All of this causes variability in movement. Learning to move your body and learning to control computers through movement is about regulating that variability.

The movement of end-effectors may be coordinated in two principled ways. One is called *open loop*, referring to a system without a feedback loop influencing the movement. When you quickly press a blinking button or grab your coffee mug, you are doing open-loop control. Conversely, in a *closed loop*, feedback is received and incorporated during motion. This feedback allows you to correct your movements, making them more precise. We discuss these control principles in Chapter 17.

In HCI, we care mostly about *aimed movements*, such as the movement of a user attempting to move an end-effector (say, their finger) to a certain location (say, a button on a touchscreen). However, a large class of non-aimed movements also has applications in HCI. Gestures are one example (say, swiping); walking is another.

More formally, aimed movements are movements in which success is defined by external constraints. To elicit the right command, we need to move the body in the right way and at the right time. For example, to send character "a," a sufficiently small body part must land exactly on the cap of the button with sufficient (but not excess) force. Except for brain–computer interfaces, most UIs are operated by aimed movements. Two fundamental types of *movement constraints* can be distinguished.

Spatially constrained aimed movements: These movements are restricted at the end or during the movement to a specified region or point. *Discrete aimed movements* are movements toward spatially bounded targets. An example is moving a mouse cursor on top of a button to select it. This movement type is prevalent in HCI. It is also one of the prime paradigms used to communicate intentions and commands to a computer; consider buttons, widgets, links, icons, and so on. By contrast, *continuous aimed movements* require the user to keep the control point within a bounding box during the entire duration of the movement. For example, keeping a cursor within a tunnel when navigating a hierarchical menu requires continuous "steering" (Figure 4.1). We discuss steering and gesturing in the next two sections.

Temporally constrained aimed movements: These movements are necessary when a target must be hit within a defined period. The goal may be to hit a target during a specific interval or to hit as close to the target as possible. An example of the former is jumping over obstacles in a video game; an example of the latter is playing notes on a piano. These two types of constraints often occur together.

In an *interception* task, spatial and temporal demands co-occur. For example, we need to catch a moving object by (1) placing a selector on its future path and (2) pressing the button when the object is within the selector's effective region. Consider, for example, hitting a tennis ball served by the opponent or sniping an enemy player in a first-person shooter game.

A fundamental tendency holds for virtually all aimed movements. The *speed–accuracy trade-off* limits the motor system, which cannot be both fast and accurate at the same time. If you think about moving your hand toward a target, it is clear that you can either move your hand quickly toward the target or be precise when hitting it. While you can carry out an action that balances these two objectives, you cannot optimally achieve both objectives at the same time.

4.2 Target acquisition

Target acquisition is a discrete, spatially constrained aimed movement. *Pointing* is perhaps the most typical target acquisition task in HCI. Here, the goal is to move the control point on top of an area denoting the target. If the control point misses the area, the movement is considered to

have missed the target. Typically, there are two performance objectives: (1) be as fast as possible and (2) do not miss the target, that is, keep the error rate below a set threshold.

While pointing is the most common target acquisition task in HCI, three other tasks are also widely studied in the literature:

- **Point target:** The target is defined as a point in space. The user's accuracy is measured as the Euclidean distance of the end point from the point selected by the user. This is practical for targets with diffuse shapes, such as a moving game character on a display that changes its shape.
- **Line/surface target:** The target is a line in space, and the user's goal is to move the control point to cross the line. Performance is measured in terms of movement time. When the target is a line segment (i.e., with finite length), accuracy can be measured as hit/miss. For example, we can define items in a menu as lines as opposed to regions to make them faster to select.
- **Postural/angular target:** Joints must be rotated to a particular angle. For example, in some dancing games, users need to replicate a specific posture shown on the display.

Two immediate questions in HCI are when these tasks are difficult to do and how we may design them to be simpler. Fitts' law provides answers to these questions.

4.2.1 Fitts' law

The time it takes a user to point to a *target*, for example, a 2D button, depends on how large it is and how far away it is. The average movement time in this task can be mathematically predicted using Fitts' law. Fitts' law is a mathematical model for predicting the average movement time required to hit a target along a one-dimensional path, which is assumed to be proportional to the difficulty of hitting the target. This difficulty is affected by the distance to the target and the width of the target.

Fitts' law is not a law in the sense of a natural law; it is a statistical regularity that was discovered through experimentation. Fitts [243], an American psychologist interested in human performance, used what is now known as the reciprocal task paradigm. Participants were asked to select targets of varying distances and widths by alternatively pointing to the left and to the right. Figure 4.2 illustrates this reciprocal task paradigm as it was used in the original experiment.

For such a task, Fitts' law states that the average movement time MT can be predicted by a linear relationship [505]:

$$MT = a + bID, \tag{4.1}$$

where a and b are regression coefficients and ID is the *index of difficulty*. The ID is an encoding of how far away the target is and how large it is, and it is defined as

$$ID = \log_2\left(\frac{D}{W} + 1\right), \tag{4.2}$$

where D is the *distance* to the target and W is the *width* of the target. The ID is easy to interpret: The closer the target is, the lower D is, and hence the lower ID is. Similarly, the larger the target is, the larger W is and thus the lower ID is. In other words, targets that are either close, large, or both have a low ID; targets that are far away, small, or both have a high ID.

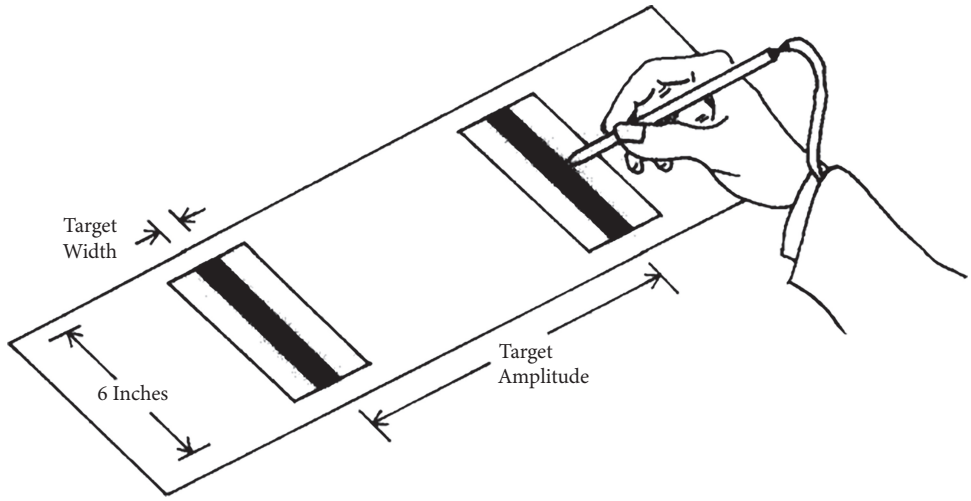

Figure 4.2 Fitts' experimental setup for manipulating movement *D* and target width W in a reciprocal tapping task. The two metal plates were to be hit with a stylus in an alternating sequence as fast as possible. Figure adapted from [505].

Another intuition from $\frac{D}{W} + 1$ is that it denotes how many targets of width W can fit within a distance *D*. In other words, *ID* denotes how many targets could have been selected, even if only one was selected in the end. We add 1 to the fraction to account for the fact that pointing aims at the center of a target, which means we have to add 0.5 W to each end of the movement. By convention, the *ID* for pointing tasks is nearly always defined in terms of logarithm base 2, which means *ID* has a unit of *bits*. The more bits that are required to express the $\frac{D}{W}$ relationship, the higher the *ID*. This intuition is discussed in more detail in Chapter 17, where we discuss the relationship between this law and information theory.

Mathematically, Equation 4.1 is identical to the equation of a straight line, $y = a + bx$, where a is the intercept and b is the slope of the line. This is because Fitts' law describes a linear relationship. The variables a and b are regression coefficients, as the intercept and slope of a line under Fitts' law are determined experimentally. By varying the distances and widths of the targets and measuring the average movement times, it is possible to infer the intercept a and the slope b of the line using a mathematical technique known as linear regression. This is what Fitts did in his 1954 paper when he discovered this regularity.

4.2.2 Using Fitts' law to assess input performance

The model has another interesting property: It can be used to compare users' performance across conditions with different target properties. Consider, for example, wanting to compare a joystick and a touchpad for target selection; how would you do that? Throughput can help us achieve this. Several definitions of throughput exist, as none is ideal. One definition of throughput is [505]

$$TP = \frac{ID_{\text{avg}}}{MT_{\text{avg}}}. \tag{4.3}$$

The downside of Equation 4.3 is that it relies on an arbitrary average *ID*. An alternative definition of throughput is given by Zhai [921]:

$$TP = \frac{1}{b}, \tag{4.4}$$

which relates throughput solely to the slope b of a line under Fitts' law. The downside of Equation 4.4 is that it ignores the effect of the intercept a. Using either definition of throughput, it is possible through experimentation to determine the throughput of various pointing methods. A higher throughput is better.

The parameters a and b may vary depending on the task, user group, and use context; there is a rich body of research investigating how these parameters may change. Also, several extensions to the model have been considered in the HCI literature, such as modeling 2D target selection [506] and targets that move on the screen.

4.2.3 Worked example

Let us show in more detail how to apply Fitts' law. The experimental task in Fitts' work, shown in Figure 4.2, is called a *reciprocal tapping task*. One can arrange this study protocol relatively easily to obtain a model under Fitts' law. In this task, users are asked to tap two targets of width W placed at distance D from each other.

In the original study, Fitts systematically manipulated D and W—the distance and width of the target, respectively. His data are reported in Table 4.1 [243], which tabulates the average *MT* as a function of D and W. Fitts noted that while there is no obvious relationship between *MT* and either D or W, they can be combined into a single term. This is the basis of Fitts' law.

Why are tasks with higher *ID* motorically harder? You can approach this question by thinking about what happens when the distance increases and when the width decreases. Both increase *ID* and therefore *MT*. In other words, all other things being equal, targets that are farther away

Table 4.1 Data from Fitts' original stylus pointing task. The predicted *MT* is $12.8 + 94.7ID$, where $ID = \log_2(2D/W)$.

MT	*D*	*W*	*ID*	Predicted *MT*
180	2	2	1	107
212	2	1	2	202
203	4	2	2	202
281	2	0.5	3	297
260	4	1	3	297
279	8	2	3	297
392	2	0.25	4	392
372	4	0.5	4	392
357	8	1	4	392
388	16	2	4	392
484	4	0.25	5	486
469	8	0.5	5	486
481	16	1	5	486
580	8	0.25	6	581
595	16	0.5	6	581
731	16	0.25	7	676

or smaller are harder to hit, and, therefore, it takes longer to hit them. Thus, movement time is related to the inverse of spatial error. Using *ID* also makes computations with the model simpler. Instead of the nonlinear relationship, we can now deal with a simpler, linear relationship.

Fitts' model based on the obtained data is shown in Figure 4.3. Averaging the *MT* data obtained in each *ID* condition, and fitting the empirical parameters *a* and *b*, Fitts found the relationship in Equation 4.1. After computing *ID*, parameters *a* and *b* are estimated,[1] yielding in this case:

$$MT = 12.8 + 94.7 \times ID, R^2 = 0.967. \tag{4.5}$$

The fit of the model, indicated by R^2, is high. R^2 is a statistical measure that indicates how much of the variance in *MT* can be explained by the regression model. Paper Example 4.2.1 shows how to compute movement time taking into account user's precision in pointing.

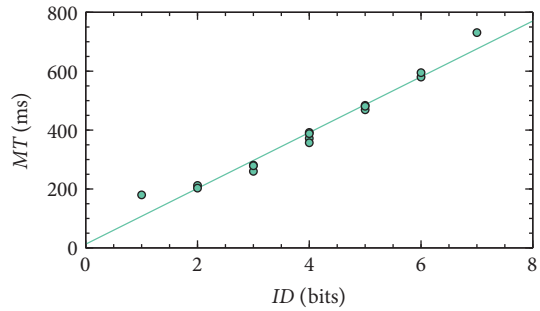

Figure 4.3 A model of stylus pointing consistent with Fitts' law. The plot shows the observed movement time *MT* versus the index of difficulty *ID*. The linear trend depicts the model $MT = 12.8 + 94.7 \times ID$. The model fit is $R^2 = 0.967$. The data are reported in Table 4.1.

Paper Example 4.2.1: Effective width

So far, we have assumed that *W* is defined by the interface or a researcher. Another way to look at it is that the user can affect it. A precise user may be more accurate than the designed target, and vice versa, and an imprecise user may be unable to reliably hit the designed target because the effective width is too large. To account for this variance, MacKenzie [505] provided a variant of the model:

$$MT = a + b \log_2 \left(\frac{D}{W_e} + 1 \right), \tag{4.6}$$

where W_e is the *effective width*. It refers to the empirical spread of end points around the target center. While *W* refers to the actual width of the target, W_e is the target that users *can* hit most of the time, or its *effective* width. You can consider the effective width to be the motor variability we can measure when a user is repeatedly carrying out the motor task.

The effective width can be computed if we can collect data on the end points of movements; that is, where the users' aimed movements end. When the end points are normally distributed,

continued

[1] Parameters *a* and *b* can be fitted to data with a statistical method called ordinary least squares, which is included in most statistical packages.

Paper Example 4.2.1: Effective width *(continued)*

the standard deviation can be used to determine W_e. The typical cut-off is $\sigma = 4.6$, which corresponds to 4% of the end points. In other words, the effective width W_e determines the width of a target that would be hit 96% of the time. The idea is shown in Figure 4.4. Because the cut-off defines the acceptable proportion of errors, it should be decided case-by-case.

This formulation clarifies the relationship of the model with the speed–accuracy trade-off. If the user tries to be faster, the target width may not change, but the effective width will increase. When *ID* increases—in other words, when the task becomes harder—either the noise (effective width) increases or the user will need to be slower.

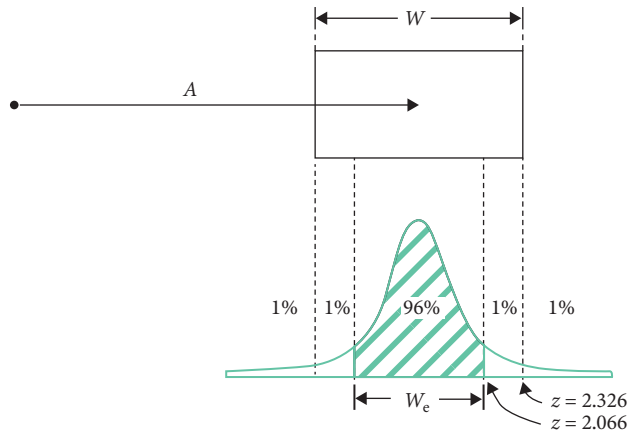

Figure 4.4 The effective width denotes the target size that the user would be able to hit X% of the time, usually set to 96%. The effective width is computed by assuming that the end points are normally distributed and setting a cut-off based on the standard deviation. Image source: Mackenzie [505].

4.2.4 Applications of Fitts' law

Fitts' law is widely used in user interface design and evaluation. If the *a* and *b* parameters are known, then it is possible to calculate how average movement times will change depending on the size and distance of targets. Fitts' law and its variants are used for designing better layouts, interaction techniques, and input devices. By exposing how user performance is affected by design-relevant factors, and by offering a unified account of both speed and accuracy, Fitts' law has become the core of our understanding of aimed movements. Fitts' law also provides a basis for empirical comparisons of input devices, and it has driven innovation in interaction techniques. Good examples are the key-target-resizing technique used in virtual keyboards and the layout optimization algorithms that have challenged QWERTY as the dominant keyboard layout [70].

Fitts' law permits the rigorous empirical comparison of input methods. Consider the problem of comparing two input devices. Figure 4.5 shows a plot comparing data from MacKenzie [505]. The plot shows that the stylus is better across the *ID* range, indicating that it is the better input option. If there was a *crossover* point, it would be exposed by the model, even if there was no observation at that *ID* point.

The concept of *ID* is powerful here. It collapses two parameters that describe a motor task into a single variable that is linearly correlated with *MT*. The alternative, called the naive-but-tempting approach by Zhai [921], involves measuring speed and accuracy on a pointing task. However, the comparison would be limited to the selected observation point. Conducting a study using

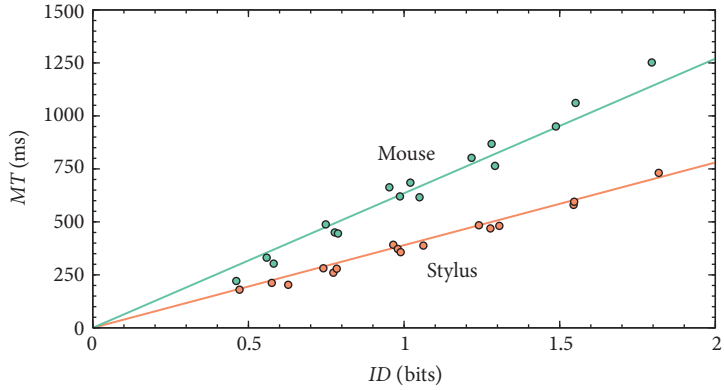

Figure 4.5 Fitts' law models allow for the comparison of user interfaces. Here, the stylus shows superior performance over the mouse. Data from Mackenzie et al. [505].

Fitts' law invites us to systematically vary D and W; the model will then provide a point of reference, the a and b parameters, for comparing the input methods across the full scope of the motor task.

Fitts' law is not limited to empirical comparisons. It can be used *analytically*—that is, prior to collecting empirical data—for the following two tasks:

- Predict the mean MT for a pointing task. However, the empirical parameters a and b must be known. They can be obtained, for example, from the literature.
- Compare pointing tasks. When a and b are equal, they can be ignored since MT will be determined by ID only.

The index of difficulty ID thus offers a handy entry point for analyzing a motor task. As we learned, increasing ID is associated with increasing MT. All other things being equal,

- an increase in D will increase MT;
- a decrease in W will increase MT;
- if D or W changes, MT can be kept constant by changing the other.

Paper Example 4.2.2: Accounting for corrective movements

Fitts' law is perhaps the most widely tested model in HCI research. It has been evaluated for many user groups, input devices, and contexts, including underwater [785]! It has weathered numerous challenges to its theoretical and mathematical assumptions. However, it says very little about what happens during movement. During pointing, we have several feedback signals available: vision, audition (the sound of movements, clicks, etc.), and proprioception. We use such signals to guide movement. Several models have been proposed that account for corrective movements.

One such variant is the *iterative corrections model* [174]. The idea is that any pointing movement consists of several *ballistic* movements with in-between corrections. A ballistic

continued

Paper Example 4.2.2: Accounting for corrective movements *(continued)*

movement cannot be modified after it is triggered; however, the next one can be planned considering sensory feedback. These redirections are corrections, hence the term "iterative corrections." Detailed recordings of how laboratory participants move showed only one or at most two corrections. Moreover, considerable variation was found in the duration of the initial submovement, violating the idea of equal durations.

An extension of the model was proposed by Meyer et al. [539]. The *stochastic optimized submovement model* defines MT as a function of not only D and W but also the number of submovements n:

$$MT = a + b\left(\frac{D}{W}\right)^{1/n}, \tag{4.7}$$

where n is an upper limit on submovements. The authors found empirically that $n = 2.6$ minimizes the RMSE.[2] Several extensions have also been proposed to compute end point variability, similar to the concept of effective width.

The iterative corrections model and the stochastic optimized submovement model assume *intermittent feedback control*. Intermittent control means that control actions cannot be carried out at any time but only after "locked" periods. In this case, the ballistic part of a motor action cannot be altered, but there is a window of time afterward to make corrections. The two models assume that such corrections are based on the error at the start of that action. Meyer's model also assumes that the neuromotor system is noisy and that this noise increases with the velocity of the submovements. This causes the primary submovement to either undershoot or overshoot the target. One known shortcoming of the model is that the number of submovements is fixed. For some given D and W, the sequence of submovements is always the same, and it is not possible to explain why the target is missed at times.

4.2.5 Limits of Fitts' law

Fitts' law is a simplification of what happens in pointing. Because it models the average MT in ID conditions, it effectively hides *variability* other than that of the resulting performance. Variability is inherent to all motor control and is affected by factors such as the movement strategy (e.g., eye–hand coordination), feedback, and the involved muscle groups. For example, if you ask a user to carry out a pointing experiment with 5–10 cm targets, the resulting model may not generalize to a setting where the targets are 10–20 cm. The model is also brittle. Even small changes in the task, user, or conditions can require the collection of a new dataset and the adjustment of the model parameters. Numerous variants have been proposed, each accounting for a specific limitation. See Paper Example 4.2.2. for one example.

4.3 Simple reactions

A *simple reaction* is another type of fundamental motor action in HCI: Something appears on the display or in the environment and the user must respond to it as quickly as possible. Figure 4.6

[2] The RMSE is the root mean square error of the distance between the model's prediction and the data. It serves as an indication of a model's fit to the data.

Figure 4.6 A *simple reaction* is a motor task where the user must respond to a prompt as quickly as possible by pressing a button. *Choice reaction* generalizes this to the case where more than one response option is available. Image by Evan Amos, used with permission.

shows an example of both simple reactions and the more general case, choice reaction. *Motor response* refers to the elicitation of a motor movement appropriate to the presented event or stimulus. This happens, for instance, when:

- reacting to a flash on the display;
- blocking an enemy's move with a counter-move in a computer game;
- answering an incoming call on a mobile device;
- braking a car;
- getting rid of an annoying dialogue box asking if you want to install a new software version.

Only one response option is available in a simple reaction—this is why it is called simple. The response can be expressed by saying something aloud, pressing a button with a finger, or even by thinking the response in brain–computer interaction. A simple reaction carries a single bit of information: a response was given. Typing a phrase, for instance, is much more complex since one must consecutively choose and select the right key from a set of at least 26 different characters. We call such choices *choice reactions*.

Simple reaction tasks have among the fastest human responses in HCI. Performance in this task is measured in milliseconds as the time duration between the onset of the event and the user's response. Typical responses, depending on the input and output modalities, are in the range of 200–250 ms. However, within this rather narrow range, relatively large differences can be observed depending on the task conditions.

Simple reactions have been studied in psychology for more than a century, and the involved cognitive processes are somewhat known. Some of the earliest studies considered the vigilance of soldiers in World War II: After sleep deprivation, or in a stressful situation, how would reaction time be altered? In HCI, simple reactions can be used to understand fast reactions, such as when playing games or musical instruments on a computer. The response consists of a decision process ("I must react") and a motor response (pressing a button). Next, we present one way of understanding this process.

4.3.1 Drift diffusion model

Ratcliff and Van Dongen [690] presented an *evidence accumulation model* that predicts the distribution of reaction times as a function of what happens after the stimulus has appeared. While it has been mostly used in safety-critical reaction tasks, such as driving, it is also illuminating for HCI applications where it is important to understand what affects the reaction time.

The idea underpinning evidence accumulation models is that the perceptual evidence for and against responding accumulates until a *threshold* is met and the motor response is launched.

- **Stimulus onset:** The event that one should respond to appears. For example, a big figure suddenly appears on the screen in a first-person shooting game.
- **Perceptual encoding:** The event is encoded as a candidate event that one should respond to. For example, the visual shape and figure of the thing that has appeared are encoded.
- **Evidence accumulation:** Every fixation samples more evidence for/against the decision to respond. In the shooting game example, "Is this a friend or a foe?"
- **Decision:** When enough evidence has been accumulated to reach the decision threshold, the corresponding motor action is launched. For example, "Yes, this is an enemy!"
- **Motor action:** The user starts an overt movement response to trigger a suitable response.

In other words, between the stimulus and the observable response, many things happen. Evidence accumulation models can account for some effects of the user interface design as well as various task-related, individual, and contextual factors. They can predict the naturally occurring variations we observe in performance when a reaction task is repeated.

The model assumes that the simple reaction RT has two sources of variation: the decision time T_d and the nondecision time T_{er}. The nondecision time is further broken down into two components, x and y. The first nondecision event is the perceptual encoding of the stimulus that lasts for some duration, marked with x. After perceiving the stimulus, a stochastic decision process starts. During this period, evidence is accumulated ("diffused") in the brain regarding whether the stimulus should be responded to. This evidence accumulation phase is affected by perceptual conditions such as noise (e.g., poor resolution, poor eyesight) and the complexity of the visual scene.

Finally, after sufficient evidence has been diffused to surpass threshold a, the decision process stops and the brain continues to the motor response process with duration y. Summing up, the average reaction time is given by

$$RT = T_d + T_{er} = T_d + x + y. \tag{4.8}$$

Figure 4.7 illustrates the model.

This model sheds light on how user interfaces can improve users' reaction time. The duration of the decision process T_d is user- and task-dependent and varies across trials. The drift rate (or the accumulation of evidence) is assumed to be normally distributed with mean v and variance η.[3] The two nondecision components x and y are summed to T_{er} and treated together in the model.

[3] The diffusion process is given by $dX(t) = vdt + sdW(t)$, where $dX(t)$ is the change in the accumulated evidence X for a time interval dt, v is the drift rate, and $sdW(t)$ are zero-mean random increments with infinitesimal variance s^2dt.

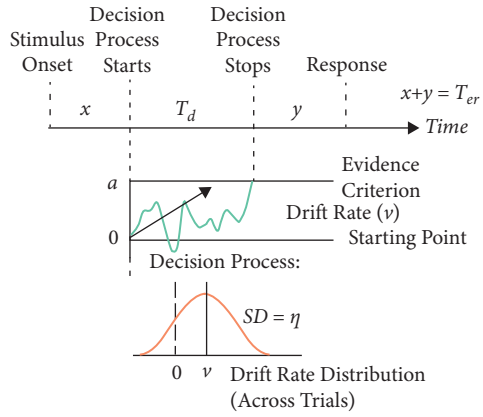

Figure 4.7 The model of Ratcliff and Van Dongen [690] defines simple reaction as consisting of a decision task and two nondecision tasks. After perceiving the prompt, a decision task starts. This is assumed to be a stochastic diffusion process where evidence accumulates toward the threshold *a*, at which point the response is emitted.

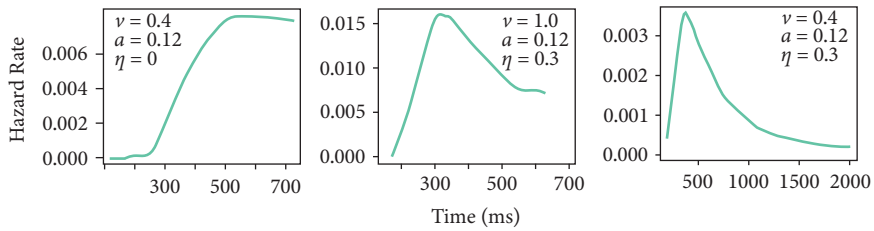

Figure 4.8 Three hazard function examples. Note the different scales of the axes. The figure is adapted from [690].

They can also change according to the user interface. For example, an auditory prompt may take longer to register than a simple visual symbol. This nondecision component is also assumed to vary across trials with standard deviation (*SD*) s_t.

The standard way of plotting the predictions is through the *hazard rate function*. It gives the probability that the decision process terminates in the next instant of time, given that it has survived up to that time. Formally, $h(t) = f(t)/(1 - F(t))$, where $f(t)$ is the probability density function and $F(t)$ is the cumulative density function. Figure 4.8 shows three examples that assume the same decision threshold a:

- **Slow but perfect responder:** In the top figure, the drift rate v is mediocre (0.4) with no variation ($\eta = 0$). The shape of the function is perfect in the sense that it is only achievable by a user who can decide their response to the stimulus with no hesitation.
- **Fast responder:** Here, the drift rate v is higher, but there is more variation ($\eta = 0.3$). This yields a much faster response, peaking at around 300 ms.
- **Slow responder:** Here, the drift rate v is mediocre (0.4), but there is still some variation ($\eta = 0.3$). The hazard distribution has a long tail. This kind of variation could be produced, for example, by sleep deprivation or by a noisy, hard-to-interpret display.

Consider an application for controlling an avatar in a 3D world. A user controls the avatar from an egocentric perspective and has to accelerate, decelerate, and steer the avatar. If the user is asked to speak on a phone simultaneously, their response times to abrupt events will be higher. This result can be attributed to the reduced drift rate. When speaking on the phone, their sampling rate slows down, and, therefore, they take longer to respond to events.

4.4 Choice reaction

In a *choice reaction task*, instead of having only one response option like in a simple reaction, n options are available. When a cue (stimulus) appears, the user must execute the *corresponding* response as quickly as possible by pressing the correct key. Each cue is associated with a single response; cues can appear with different probabilities. The fingers are supposed to rest on the possible keys to minimize the effect of pointing on the response. Consider this example: You are playing a racing game and your car is approaching a T-crossing at a fast pace. Which way should you turn? This is a two-alternative forced-choice (2AFC) task.

Performance in choice reaction tasks is measured by the *choice reaction time* (*CRT*). It is the time that has elapsed between the presentation of the cue and the response. Errors—choosing the wrong response or not responding—can be dealt with by insisting on a correct response or by allowing errors to happen and reporting the error rate alongside the *CRT*.

Choice reaction is a generalization of a simple reaction. When $n = 1$, we have a simple reaction. When $n = 2$, we have a 2AFC task where there are two response options and only one can be chosen. 2AFCs are common in HCI. For example, a "restart now" system prompt forces the user to pick between the options yes and no. When n increases above 2, we find an interesting and practically important relationship between n and *CRT*.

4.4.1 The Hick–Hyman law

The Hick–Hyman law is a statistical relationship between n and *CRT* that was discovered independently by Hick and Hyman [340, 369]. The law states that when the number of response options increases, the *CRT* increases. Trying to be faster than what the law suggests will lead to errors; allowing more time for responding will result in fewer errors.

Formally, given n equally probable response options, the average *CRT* follows approximately:

$$CRT = a + b \cdot \log_2(n), \tag{4.9}$$

where a and b are empirical constants determined by fitting a line to the data. In cases with uncertainty about whether to respond or not, a value of 1 can be added to n:

$$CRT = a + b \cdot \log_2(n + 1). \tag{4.10}$$

Do-not-respond is just one more option.

The parameter b controls how strongly the increase in n affects *CRT*. Figure 4.9 shows two examples.

Why is there a *binary* logarithm in the standard formulation? One interpretation is that it reflects a binary search on the part of the user. When a cue appears, the user first picks half of the options and rejects the other half, and then picks half of the remaining options, and so on, until they finally

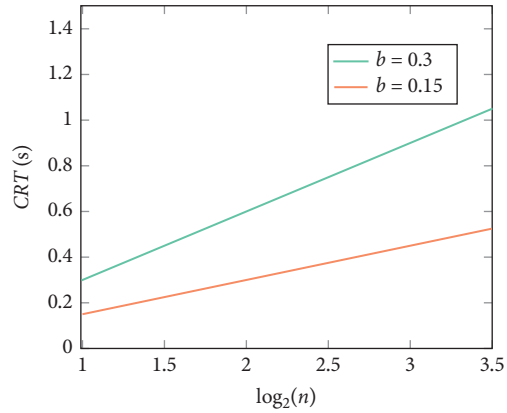

Figure 4.9 Parameter *b* controls how the number of options affects *CRT*.

identify the correct response. This process also links Hick–Hyman law to information theory [754]; see Chapter 17.

Another interpretation of the law is that it denotes the *uncertainty about the stimulus*. In the case of choices with unequal probabilities, the law can be expressed as

$$CRT = a + bH, \tag{4.11}$$

$$H = \sum_{i}^{n} p_i \log_2(1/p_i + 1), \tag{4.12}$$

where p_i is the probability of response option i. Even though there is a large number of options available, if only a small subset is considered (i.e., the sum of their probabilities is close to 1), *CRT* can be low.

4.4.2 Applications in HCI

Both the Hick–Hyman law and Fitts' law were introduced to HCI by Card et al. [129] as motor control principles for improving the usability of user interfaces. However, the Hick–Hyman law has had fewer applications than Fitts' law. Why?

To answer this question, note that the Hick–Hyman law states that less is better, which appears trivial; if you design an interface with fewer possible responses, users will be quicker in responding to it. However, if you need to decide how to show n elements on a display, the law predicts that there is a benefit from showing *all* the elements at once. Consider the design example in Figure 4.10. Because of the logarithmic term, the best design, contrary to our intuition, is Design (a). Hence, the design principle should be "more is better." However, when other factors are considered, the situation is not so simple. There are also benefits from pagination and hierarchy, which are not governed by the Hick–Hyman law. Information foraging theory explains that well-organized hierarchies help users save time by *skipping* whole sections of elements (Chapter 21). Visual search and pointing also take more time as the number of elements increases.

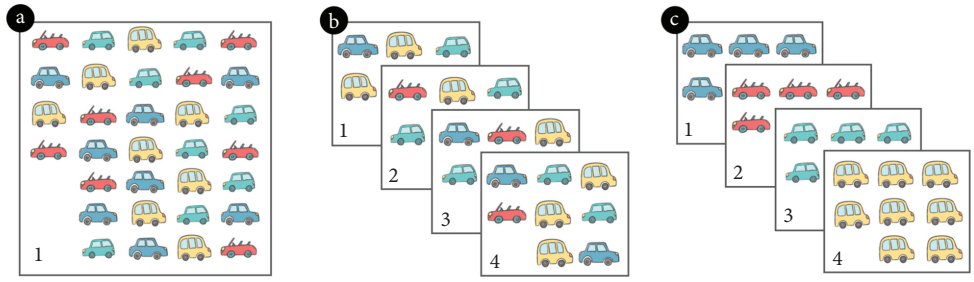

Figure 4.10 You need to show 32 items to the user. According to the Hick–Hyman law, which of the designs is the fastest to use? Figure from Liu et al. [490].

The second form of the law we discussed, which defines *CRT* as a function of entropy *H*, implies that decreasing uncertainty improves performance. How can we exploit this in practice? *Stimulus–response compatibility* has a strong effect on *CRT*. When stimuli and responses are compatible, they are ordered or otherwise structured in a consistent way. Some simple mapping exists; for example, cues have the same spatial order as responses. This means that the response should be similar to the stimulus itself, such as turning a steering wheel to turn the wheels of the car. The action the user performs is similar to the response the driver receives from the car. Liu et al. [490] showed that in HCI tasks where compatibility is high, the Hick–Hyman slope almost flattens out. It is generally desirable to find consistent mappings between stimuli and responses.

Design and training also have an effect on the slope *b* and intercept *a* of the model. The Hick–Hyman law governs the novice-to-intermediate range of performance. When the user has had extensive practice in responding to the task, the slope decreases, eventually flattening out. With thousands of practice trials on the same task, the response time can effectively be constant when *n* is lower than 10 [560].

One should be careful when applying the law to cases where $n > 10$. In its basic interpretation, the *n* end-effectors are supposed to rest on the *n* keys associated with the *n* responses. When $n > 10$, other factors come into play, such as moving fingers, finding targets visually, and reasoning.

4.5 Gesturing

Gestural interfaces use continuous shapes as input. Consider, for example, handwriting as text input: For a letter to be recognizable by the decoder, its shape must have certain segment lengths and curves.

4.5.1 Crossing

A *crossing* task is a task that relaxes the stopping constraint in Fitts' law, that is, the *D* parameter of the target [7].

Recall that Fitts' law describes a one-dimensional pointing action. Hence, when a user is attempting to hit a target, the user needs to both move the pointer toward the target *and* stop the pointer before leaving the target. By contrast, in a crossing task, the user does not need to stop at the target.

Instead, the user needs to ensure the target is crossed. In this case, the width parameter W refers to the size of the target the user is crossing.

Through experimentation, it has been established that crossing tasks follow a similar statistical relationship as Fitts' law and can be described using the equation from this law:

$$MT = a + b \log_2 \left(\frac{D}{W} + 1 \right). \tag{4.13}$$

Unlike pointing, crossing actions can be chained together, allowing the user to cross multiple targets in one motion. An example of a system leveraging crossing actions is CrossY [26], a drawing program that allows the user to input commands via crossing actions with a pen.

4.5.2 Steering

A *steering* task is a task where the user is moving a cursor through a form of tunnel constraint, as proposed by Accot and Zhai [6]. It has a relationship to Fitts' law (Paper Example 4.5.1).

In general, the time T it takes a user to steer a cursor through a tunnel is

$$T = a + b \int_C \frac{ds}{W(s)}, \tag{4.14}$$

where a and b are empirically determined parameters, C is the tunnel constraint parameterized by s, and $W(s)$ is the width of the tunnel at s. As with Fitts' law and the crossing law, the parameters a and b may vary depending on the user group, task, and use context.

It is possible to define an index of difficulty for steering tasks. In this case, the index of difficulty is directly related to the parameterized curve C:

$$ID_C = \int_C \frac{ds}{W(s)}. \tag{4.15}$$

By differentiating both sides of Equation 4.14 with respect to s, we obtain

$$\frac{ds}{dt} = \frac{W(s)}{b}, \tag{4.16}$$

where we observe that, as expected, the instantaneous movement speed $\frac{ds}{dt}$ at point s in the tunnel is proportional to the width $W(s)$ of the tunnel at that point.

Steering law can be used to model users moving cursors within tunnels, such as a user moving a mouse pointer along a hierarchical linear pull-down menu structure.

Paper Example 4.5.1: Relationship between Fitts' law and steering law

There is a mathematical relationship between steering law and Fitts' law [6]. A steering task with a single goal constraint on each end follows the same logarithmic relationship as Fitts' law:

$$ID_1 = \log_2\left(\frac{D}{W} + 1\right). \tag{4.17}$$

This task can be extended by adding a single further goal constraint, which yields the following relationship:

$$ID_2 = 2\log_2\left(\frac{D}{2W} + 1\right). \tag{4.18}$$

Note that as the number of goal constraints increases, the user has to be more careful in ensuring they pass through all the goal constraints.

Then, by generalizing the number of goal constraints the user has to pass through to N, the goal constraints form a tunnel constraint:

$$ID_N = N\log_2\left(\frac{D}{NW} + 1\right). \tag{4.19}$$

With the limit $N \to \infty$, we obtain the following relationship:

$$ID_\infty = \frac{D}{W\ln 2}. \tag{4.20}$$

Note that in the limit, the index of difficulty is no longer related to $\log_2\left(\frac{D}{W}\right)$ but directly to $\frac{D}{W}$. In other words, in an N goal passing task, as N approaches infinity, the difficulty in achieving the task is no longer related to the logarithm of the distance W and the width W. This explains why the index of difficulty for a steering task in Equation 4.15 lacks a logarithmic relationship.

4.5.3 Viviani's power law of curvature

The models discussed thus far in this chapter aim to predict the task completion time. One limitation of steering law is that it does not account for complex shapes or changing shapes. Consider, for example, tracing a shape or drawing. What happens *during* the movement? If we wish to understand gestures, we need to understand where the curvature changes.

Viviani's *power law of curvature* (PLoC) is a kinematics model for smooth curved trajectories [858]. *Kinematics models* cover aspects of motion during pointing, such as position, velocity, acceleration, or jerk, without consideration of the time-varying phenomena that produce them. They predict the moment-by-moment motion or its properties, such as the radius of curvature or the tangential velocity at any point. This makes kinematics models useful for modeling gestures.

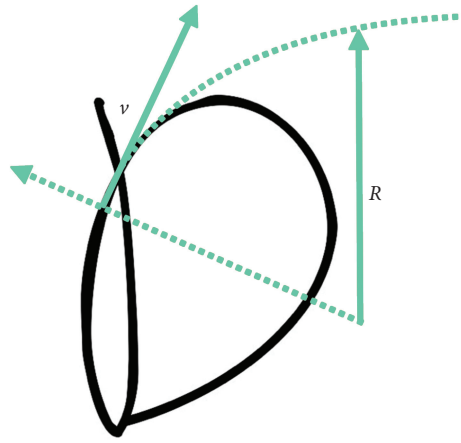

Figure 4.11 Try drawing the letter "d" with a stylus or pen. Viviani's power law of curvature predicts the momentary velocity v when drawing such smooth curved trajectories. R is the radius of the curvature.

PLoC pertains to handwriting and drawing behavior, particularly when the trajectories are smooth (i.e., without sharp corners). PLoC relates the radius of curvature $r(s)$ at any point s along the trajectory with its corresponding tangential velocity $v(s)$:

$$v(t) = kr^{\beta}, \tag{4.21}$$

where k is an empirical gain factor and β is an empirical parameter. The model indicates that the larger the curvature, the slower the motion of the end-effector at that point.

Next, the total time for a full segment S, assuming a smooth curvature without corners, can be computed as

$$T = \frac{1}{k}\int_0^S r(s)^{-\beta} ds. \tag{4.22}$$

For example, in a study using a stylus as the input device, $K = 0.0153$ and $\beta = 0.586$ [125]. The model has achieved a high fit with empirical data in drawing, with and without visual guidance.

What happens if the movement is physically larger or smaller? *Isochrony* refers to the empirical observation that the average velocity of movements increases with distance [858]. Thus, movement distance is a weak predictor of movement time in a trajectory. Users simply cover larger distances faster. Viviani's PLoC has been shown to cover isochrony. The PLoC-like pattern has been argued to be due to pattern generators in the neuromotor system that operate in an oscillatory fashion [737].

Summary

- Motor control is necessary for users to perform actions in a user interface.
- The time it takes users to perform actions can be predicted for some fundamental activities, such as pointing, crossing, steering, and reacting to stimuli.

Exercises

1. Applying Fitts' law. Given the following target widths and distances, calculate the average movement times for all combinations using Fitts' law. Comment on the validity and implications of the calculated average movement times: D = [0.01, 0.05, 0.1, 1, 100] m; W = [0.01, 0.050.1, 1, 100] m.
2. Keypad design. Considering Fitts' law, think about a layout of 3×3 icons, for example, a numeric keypad. What happens to the movement time when you make the icons bigger? How much faster will the interface be if you arrange the icons in a list (1×9) instead of a 3×3 layout? Could you do anything else to reduce the movement time?
3. Applying the Hick–Hyman law. Use the Hick–Hyman law to calculate the *CRT* for an interface with four options, where the probability of each option is 0.1, 0.1, 0.1, and 0.7, respectively. Then, recalculate the *CRT* assuming the probability of each option is 0.25. Compare and comment on the results.
4. Understanding menu navigation. Consider a cascading linear pull-down menu. The user wishes to select a menu item in a submenu. In one variant, the user has to steer the cursor through a top-level submenu item to trigger the submenu. When the cursor is not over the top-level submenu item, the submenu instantly disappears. This type of implementation is common in web interfaces. In another variant, the submenu triggers when the user steers the cursor to the top-level submenu item and the submenu stays in place even when the user moves the cursor away from the top-level submenu item. Comment qualitatively on the type of movement time model that is suitable for the two types of menus and which design would be preferable and why. Optionally, consider how the preferable submenu trigger could be implemented in the simplest way possible.
5. Comparing input devices. Carry out a controlled pointing study with two input devices of your choice, fit the free parameters (a, b) of Fitts' law, and plot the resulting models. To collect the data, you can use an implementation of the task on the web, such as http://simonwallner.at/ext/fitts/. Tip: Pay attention to the ranges of D and W you pick.
6. Beating Fitts' law. User interfaces make it possible to do things to targets, such as changing their size or distance, that are impossible in physical environments. Given Fitts' law, ideate interaction techniques that would facilitate pointing (i.e., decrease MT). Tip: Ravin Balakrishnan discussed relevant techniques in a classic paper titled "Beating" Fitts' law: https://www.sciencedirect.com/science/article/pii/S107158190400103X.
7. Use the drift diffusion model (code available via the book's homepage) to estimate a gamer's reaction time in two conditions: good health and after sleep deprivation. To this end, you need to think about which process in the model is affected by sleep deprivation.